PENSE-BÊTE · TECHNIQUE ET COMMANDES

Architecture d'Automatisation Linux

06-310 PSE 25-27 3-2 Architecture d'Automatisation Linux - Du Script au Modèle Déclaratif — source : fiche de révision du module (gemma4, 03/11/2026)

À RETENIR

1. L'approche procédurale par scripts isolés se heurte à des limites de scalabilité, de variance et de traçabilité.
2. La politique déclarative remplace les instructions par la description d'un état final désiré pour gérer la complexité.
3. L'architecture peut utiliser un modèle push via un contrôleur central ou un modèle pull avec des agents sur les cibles.

Les limites de l'approche procédurale (scripts isolés)

Limite	Conséquence
Loi d'échelle	L'approche devient inefficace quand la taille de l'infrastructure augmente significativement
Mur de la variance	Impossibilité de prévoir toutes les différences d'environnement ou configurations résiduelles
Mur de l'idempotence	Scripts standards provoquant erreurs ou duplications lors de réexécutions
Mur de la traçabilité	Gestion chaotique par absence de source de vérité unique
Mur de l'action humaine	Délais de réparation dépendants de la disponibilité et de la vitesse d'un technicien

Bénéfices de l'architecture d'automatisation

Notion	À retenir
Politique déclarative	Remplace les instructions par la description d'un **état final désiré** ; l'outil gère la complexité et les variations d'environnement.
Collaboration et traçabilité (GitOps)	Référentiel centralisé : partage des connaissances et **validation systématique** avant production ; l'historique est un **sceau infalsifiable** (audits, **retours en arrière** fiables).
Fonctionnement proactif	**Convergence continue** qui corrige les dérives avant la panne ; les équipes n'interviennent plus que sur les cas complexes et la définition des politiques.

Fondations d'une architecture robuste

Notion	À retenir
Gestion d'état et idempotence	Le moteur compare l'**état observé** avec l'**état spécifié** (fichiers de référence) pour assurer l'**auto-guérison** et des reprises sans risque de corruption.
Gestion sécurisée des secrets	Aucun mot de passe ni clé d'API **en clair** dans le code ; secrets **injectés dynamiquement** à l'exécution par des outils spécialisés (rotation facilitée).
Configuration centralisée	Un **annuaire distribué** fournit paramètres dynamiques et **découverte de services** sans redéployer les règles d'automatisation.

Architecture d'automatisation

Notion	À retenir
Modèles d'exécution	Modèle **push** (contrôleur central vers cibles) ou **pull** (agents sur cibles récupérant la configuration périodiquement).
Conception modulaire	Structure en **rôles indépendants** encapsulant la logique ; l'ordre est garanti par la **déclaration explicite des dépendances**.
Industrialisation	Tests de **syntaxe** et **conformité** via **pipelines d'intégration continue** ; sécurisation par **bastion** et **moindre privilège** ; observabilité par **centralisation des journaux** et **métriques**.

Pièges et confusions

- ⚠ Confondre l'approche procédurale avec la politique déclarative qui décrit l'état final.
- ⚠ Oublier que la gestion d'état et l'idempotence sont les seules garanties pour un comportement prévisible.
- ⚠ Négliger l'intégration continue qui protège la production contre les erreurs humaines et mauvaises configurations.
- ⚠ Ignorer l'observabilité proactive qui permet d'identifier les machines dérivant de leur état souhaité.
- ⚠ Sous-estimer le passage à l'échelle qui transforme l'administration en ingénierie logicielle.

Sigles et définitions

IaC	Gestion et provisionnement de l'infrastructure via des fichiers de définition lisibles par machine.
GitOps	Évolution de l'IaC où le système de contrôle de version est la source de vérité unique et le déclencheur.
Idempotence	Propriété garantissant qu'une opération répétée produit le même résultat sans créer de dérive.
MTTR	Temps moyen nécessaire pour rétablir un service informatique après une panne.
Pull Request	Proposition de modification de code pour revue par les pairs avant intégration.
Drift	Écart entre l'état désiré configuré dans le dépôt central et l'état réel sur les serveurs.

Auto-test

1. Quel est le problème de l'approche procédurale face à l'augmentation de la taille de l'infrastructure ?
2. Que remplace la politique déclarative ?
3. Comment est garantie la structure en rôles indépendants ?
4. Quels sont les deux modèles d'exécution possibles ?
5. Que compare le moteur pour assurer l'auto-guérison ?

Réponses : **1.** La loi d'échelle. · **2.** Les instructions. · **3.** Par la déclaration explicite des dépendances. · **4.** Modèle push ou modèle pull. · **5.** L'état observé avec l'état spécifié.

À compléter par la fiche, le podcast et le quiz du module. Rédigé à partir de la fiche de révision ; seuils et chiffres à la date de la source.